

Git Basic Commands

```
$ git status  
$ git add  
$ git commit  
$ git push
```

Where We Are Now

From the first slide set, you should now have:

- A GitHub account
- A forked repository on GitHub
- A cloned copy on your computer

Now we will learn the basic Git commands we use every day on a NEW repo.

Why Create a New Repo?

The forked course repo is useful for practicing fork and clone.

For basic commands, we will create a small personal repo called `book` .

This gives us a clean, simple place to practice:

- editing files
- making commits
- pushing to GitHub
- pulling remote changes
- checking old commits

Today's Goal

By the end of this section, you should be able to:

- check what changed
- stage selected files
- commit a local snapshot
- push changes to GitHub
- pull changes from GitHub
- look at project history
- temporarily check out an older version

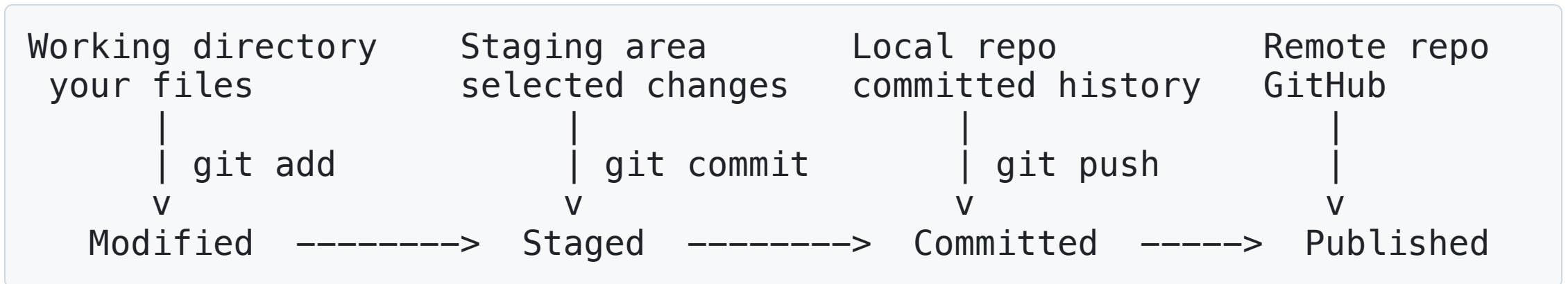
The Mental Model

Think of Git as a project workflow:

Git concept	Analogy
Working directory	Your desk
Staging area	Review tray
Local repository	Personal archive
Remote repository	Shared online archive

Desk → Review tray → Personal archive → Shared archive

The Basic Workflow



Important idea:

`git add` stages changes. `git commit` saves staged changes.

```
git status
```

git status : What Is Happening?

`git status` shows the current state of your repository.

```
git status
```

It tells you:

- which branch you are on
- which files changed
- which files are staged
- which files are untracked
- whether your local branch is ahead or behind GitHub

Analogy: `git status`

`git status` is like checking your desk and review tray.

It answers:

```
What is on my desk?  
What is already in the review tray?  
What is ready to archive?  
Is my archive synced with GitHub?
```

Use it often.

Example: `git status`

```
$ git status
```

```
On branch main
```

```
Your branch is up to date with 'origin/main'.
```

```
Changes not staged for commit:
```

```
    modified:   README.md
```

```
Untracked files:
```

```
    notes.txt
```

Plain language:

```
README.md changed.
```

```
notes.txt is new.
```

```
Neither is staged yet.
```

```
git add
```

`git add`: Move Changes to Staging

`git add` puts selected changes into the staging area.

```
git add README.md
```

This means:

```
Include README.md in the next commit.
```

It does **not** create a saved version yet.

Connection: Staged = `git add`

When a file is **staged**, it is in the review tray.

```
git add README.md
```

Desk → Review tray

You are telling Git:

This change should be part of my next saved snapshot.

Common `git add` Patterns

Stage one file:

```
git add README.md
```

Stage all current changes:

```
git add -A
```

Good practice:

```
git status  
git add README.md  
git status
```

Check before and after staging.

```
git commit
```

git commit: Save a Snapshot Locally

`git commit` saves the staged changes into your local Git history.

```
git commit -m "Update README"
```

The commit message explains what changed.

Connection: Committed = `git commit`

When a change is **committed**, it is saved in your personal archive.

Review tray → Personal archive

Important:

```
git add      = choose what to save  
git commit  = save it locally  
git push     = upload it to GitHub
```

What Makes a Good Commit Message?

A good message is short and meaningful.

Good examples:

```
git commit -m "Add project README"  
git commit -m "Fix typo in installation instructions"  
git commit -m "Add data cleaning script"
```

Less helpful:

```
git commit -m "stuff"  
git commit -m "changes"  
git commit -m "final final version"
```

```
git log
```

git log: View Project History

`git log` shows the commit history.

```
git log
```

A shorter version is often easier to read:

```
git log --oneline
```

This shows each commit as one line with a short commit ID.

Analogy: `git log`

`git log` is like opening the archive index.

It answers:

```
What snapshots exist?  
When were they made?  
Who made them?  
What message did they leave?  
What is the commit ID?
```

Example: `git log --oneline`

```
$ git log --oneline  
  
a72f91c Add notes file  
4c19b21 Update README  
91e7c03 Initial commit
```

Each line is a saved checkpoint.

The first part is the commit ID:

```
a72f91c
```

```
git push
```

git push: Send Local Commits to GitHub

After committing locally, GitHub does not automatically know about your changes.

Use **git push** to upload your local commits.

```
git push
```

Or, if needed:

```
git push origin main
```


Analogy: `git push`

`git push` sends updates from your personal archive to the shared online archive.

Personal archive	→	Shared online archive
Local repo	→	Remote repo
Your computer	→	GitHub

A commit is local first.

GitHub sees it only after you push.

```
git pull
```

git pull: Bring GitHub Changes to Your Computer

If the remote repository has changes you do not have locally, use:

```
git pull
```

This downloads and applies the remote changes to your local repository.

Analogy: `git pull`

`git pull` brings updates from the shared online archive back to your personal archive and desk.

```
Shared online archive → Personal archive → Desk  
GitHub                → Local repo       → Working directory
```

Good habit:

```
git pull
```

at the beginning of a work session.

```
git checkout <commit ID>
```

git checkout <commit ID>: Visit an Older Snapshot

You can temporarily look at an older version of your project:

```
git log --oneline  
git checkout <commit ID>
```

Example:

```
git checkout 91e7c03
```

This lets you inspect what the project looked like at that commit.

Important: Detached HEAD

When you check out a commit ID directly, Git puts you in a temporary state called **detached HEAD**.

Plain language:

```
You are visiting an old snapshot.  
You are not on the main branch right now.
```

To go back:

```
git switch main
```

or:

```
git checkout main
```

Analogy: `git checkout <commit ID>`

Checking out a commit is like taking an old snapshot out of the archive to look at it.

You can inspect it, but you should be careful about editing there.

Archive snapshot → temporary viewing table

For today:

Use checkout to look, then return to `main`.

Code-Along Exercise

Create a new GitHub repo and practice the full workflow.

Exercise Goal

We will create a new repository on GitHub and practice:

```
git clone  
git status  
git add  
git commit  
git log  
git push  
git pull  
git checkout <commit ID>
```

By the end, you will have your own GitHub repository with multiple commits.

Step 1: Create a New Repo on GitHub

On GitHub:

1. Click **New repository**

2. Name it:

book

3. Choose **Public** or **Private**

4. Check **Add a README file**

5. Click **Create repository**

Step 2: Clone Your Repo

On GitHub, copy the HTTPS URL from the green **Code** button.

Then in your terminal:

```
cd Desktop
git clone https://github.com/<your-username>/book.git
cd book
```

Replace `<your-username>` with your GitHub username.

Step 3: Check the Repository

```
git status  
git log --oneline
```

You should see:

```
You are on branch main.  
Your working tree is clean.  
There is at least one commit.
```

That first commit was created when GitHub added the README.

Step 4: Modify the README

Add a line to `README.md`.

Option A: edit the file in VS Code or another editor.

Option B: use the terminal:

```
echo "This is the Git repository of my book." >> README.md
```

Then check:

```
git status
```

Step 5: Stage the Change

```
git add README.md
```

Remember:

```
git add = move from desk to review tray  
git add = modified → staged
```

Check again:

```
git status
```

Step 6: Commit the Change

```
git commit -m "Update README with project description"
```

Remember:

```
git commit = save staged changes to local archive  
git commit = staged → committed
```

Then check history:

```
git log --oneline
```


Step 7: Push to GitHub

```
git push
```

Then refresh your GitHub repository page.

You should see your README update online.

```
Local committed change → GitHub
```

Step 8: Make a Remote Change on GitHub

On GitHub:

1. Open `README.md`
2. Click the pencil/edit icon
3. Add one more line:

```
This line was added on GitHub.
```

4. Commit the change on GitHub

Now GitHub has a commit your computer does not have yet.

Step 9: Pull the Remote Change

Back in your terminal:

```
git status  
git pull  
git log --oneline
```

Now your local repository should include the commit you made on GitHub.

```
GitHub → your computer
```

Step 10: Check Out an Older Commit

First, view history:

```
git log --oneline
```

Copy an older commit ID.

Then:

```
git checkout <commit ID>
```

Look at the files.

What changed?

Step 11: Return to `main`

After inspecting the old snapshot, return to your main branch:

```
git switch main
```

or:

```
git checkout main
```

Then check:

```
git status
```

The Daily Git Routine

Everyday Workflow

A normal Git workflow often looks like this:

```
git pull

# edit files

git status
git add README.md
git commit -m "Describe the change"
git push
```

Plain language:

```
Sync → edit → check → stage → commit → share
```

Command Summary

Command	Purpose	Analogy
<code>git status</code>	Check current state	Check desk and tray
<code>git add</code>	Stage selected changes	Put pages in review tray
<code>git commit</code>	Save local snapshot	Archive labeled snapshot
<code>git log</code>	View history	Read archive index
<code>git push</code>	Upload commits	Send to shared archive
<code>git pull</code>	Download updates	Get from shared archive
<code>git checkout <ID></code>	Visit old commit	Inspect old snapshot

Common Issues

"Nothing to commit"

There may be no saved file changes.

```
git status
```

"Rejected push"

GitHub has changes you do not have yet.

```
git pull  
git push
```

"Detached HEAD"

You checked out a commit ID.

Mini Challenge

Create a new file and publish it to GitHub.

```
echo "My Git notes" > notes.txt  
git status  
git add notes.txt  
git commit -m "Add Git notes"  
git push
```

Then refresh your GitHub repo.

Do you see `notes.txt` ?

Questions?

```
git status
```

When in doubt, start here.